

GEAD: Explainable Deep Learning-Based Anomaly Detection for Network Security

Presented by: Prajwal Pawar

Student ID: 2023UCP1598

Submitted to: Assoc. Prof. Ramesh Babu Batulla

1. Introduction

With the rapid growth of internet-connected systems, cyberattacks on enterprise networks have increased significantly. Traditional Intrusion Detection Systems (IDS) mainly rely on signature-based detection methods, which are ineffective against unknown or zero-day attacks.

To overcome these limitations, modern IDS systems increasingly use Machine Learning and Deep Learning techniques for anomaly detection. Deep learning models such as Autoencoders can learn complex network traffic patterns and detect abnormal behavior with high accuracy.

However, deep neural networks suffer from a major limitation: lack of explainability. These systems often behave as black boxes and cannot clearly explain why a particular network flow was classified as malicious.

GEAD (Generating Explainable Anomaly Detection) is a framework designed to solve this problem by combining:

- Deep anomaly detection
- Explainable AI
- Rule-based interpretation

GEAD generates interpretable decision rules from deep anomaly detection models, helping security analysts understand the reasoning behind attack detection.

This project focuses on implementing and analyzing the GEAD framework using deep learning and explainable decision tree techniques for network intrusion detection.

2. Objective

The main objective of this project is to implement and study the GEAD framework for explainable network intrusion detection.

The project aims to:

- Train an autoencoder-based anomaly detection model.
- Detect malicious network traffic using reconstruction error.
- Generate explainable regression-tree based rules.
- Identify low-confidence regions in anomaly predictions.
- Generate fine-grained interpretable security rules.
- Evaluate explainability and anomaly detection performance.
- Analyze compatibility challenges in modern ML environments.

3. Dataset Used

3.1 CICIDS2017 Dataset

The project uses the CICIDS2017 intrusion detection dataset.

The dataset contains realistic network traffic including:

- Benign traffic
- DoS attacks
- Brute-force attacks
- Web attacks
- Botnet traffic

The Tuesday traffic subset was used for training and evaluation.

The dataset contains multiple network-flow features such as:

- Flow duration
- Packet statistics
- Bytes per second
- Header information
- Connection statistics

These features help the model distinguish between normal and malicious network behavior.

4. Methodology

4.1 Autoencoder-based Anomaly Detection

An Autoencoder is a neural network trained to reconstruct input data.

In GEAD:

- The autoencoder is trained only on benign traffic.
- The model learns normal traffic patterns.
- Abnormal traffic produces high reconstruction error.

The reconstruction error is calculated as:

- Reconstruction Error = $|X - \hat{X}|$

Where:

- X = original input
- $\hat{X}$ = reconstructed output

If reconstruction error exceeds a threshold, the traffic is classified as anomalous.

4.2 Root Regression Tree Generation

GEAD generates a regression tree to approximate the behavior of the neural anomaly detector.

The regression tree:

- Mimics neural network decisions
- Produces interpretable IF-THEN rules
- Improves explainability

Example rule:

- IF flow duration > threshold
- AND bytes/sec < threshold
- THEN anomaly

4.3 Low-confidence Leaf Detection

Some regions of the regression tree contain uncertain predictions.

These regions are called:

- Low-confidence leaves

They represent areas where:

- Neural network predictions are complex
- Tree approximation is weak

GEAD identifies these regions for further refinement.

4.4 Data Augmentation

GEAD generates additional synthetic samples around low-confidence regions.

Purpose:

- Improve explainability
- Improve local decision boundaries
- Better approximate neural network behavior

Data augmentation increases training samples significantly.

4.5 Fine-grained Rule Generation

For each low-confidence region:

- Local subtrees are generated
- Detailed interpretable rules are extracted

These fine-grained rules:

- Improve fidelity
- Improve interpretability
- Help explain anomaly decisions clearly

5. Model Pipeline

The complete GEAD pipeline is:

```
None
Network Traffic Dataset
↓
Feature Extraction
↓
Data Preprocessing
↓
Feature Normalization
↓
Train / Validation / Test Split
↓
```

```
Autoencoder Training
↓
Reconstruction Error Computation
↓
Anomaly Detection
↓
Root Regression Tree Generation
↓
Low-confidence Leaf Detection
↓
Data Augmentation
↓
Fine-grained Subtree Generation
↓
Interpretable Security Rules
```

6. Data Preprocessing

The following preprocessing steps were applied:

- Missing value handling
- Feature normalization
- Label encoding
- Train-validation-test split
- Benign-only training selection

Feature normalization improves:

- Neural network convergence
- Stable optimization
- Consistent feature scaling

7. Experimental Setup

Development Environment

Component	Technology
Platform	Google Colab
Programming Language	Python 3.12
Deep Learning Framework	PyTorch
ML Library	Scikit-learn
Data Processing	NumPy, Pandas
Visualization	Matplotlib

8. Experimental Results

8.1 Autoencoder Performance

The autoencoder successfully learned benign traffic patterns using reconstruction-error based learning.

The model generated anomaly scores capable of separating normal and malicious traffic.

Validation Performance:

```
roc_auc: 0.7528284049379022
```

```
Performance on Validation Set:
```

```
TPR: 0.9973583798062519 FPR: 0.4036513033098607
```

```
TN: 64448 TP: 3398 FP: 43623 FN: 9
```

8.2 Root Regression Tree Results

The generated regression tree produced:

- 41 leaf nodes
- Maximum depth = 12
- 24 low-confidence leaves

These statistics demonstrate successful explainability-tree generation.

8.3 Data Augmentation Results

GEAD generated augmented explainability samples:

22611→20143022611→20143022611→201430

This improved local rule learning around uncertain regions.

8.4 Fine-grained Rule Generation

GEAD successfully generated:

- 47 interpretable local security rules

These rules approximate neural network behavior in human-readable form.

9. Visualizations Generated

The following visualizations were successfully generated:

- ROC Curve
- Reconstruction Error Distribution
- Regression Tree Statistics
- Low-confidence Leaf Analysis

These visualizations help analyze:

- Detection performance
- Explainability quality
- Decision boundary behavior

10. Challenges Faced

During implementation, compatibility issues were encountered between:

- Modern Python 3.12
- New sklearn tree internals
- Original GEAD implementation

Specific issues included:

- Deprecated NumPy attributes

- Pandas compatibility changes
- sklearn tree serialization incompatibilities

Despite these issues, the major explainability stages of GEAD were successfully reproduced.

11. Advantages of GEAD

- Combines deep learning with explainability
- Generates interpretable security rules
- Improves analyst trust
- Detects unknown attacks
- Enhances forensic analysis
- Supports security compliance requirements
- Maintains strong anomaly detection performance

12. Limitations

- Final merged-tree reconstruction faced compatibility issues
- Deep learning training is computationally expensive
- Large datasets require significant memory
- Explainability-tree generation increases processing overhead
- Framework depends heavily on sklearn internals

13. Future Work Scope

Possible future improvements include:

- Real-time IDS deployment
- Modern sklearn compatibility support
- Online anomaly learning
- Integration with enterprise SIEM systems
- Explainability optimization
- Deployment on cloud-based IDS platforms
- Hybrid transformer-based anomaly detection

14. Technologies Used

Component	Technology
Programming Language	Python
Deep Learning Framework	PyTorch
ML Framework	Scikit-learn
Dataset	CICIDS2017
Visualization	Matplotlib
Notebook Environment	Google Colab

15. Conclusion

This project successfully implemented the GEAD framework for explainable anomaly detection in network security.

GEAD combines:

- Deep anomaly detection
- Explainable regression trees
- Fine-grained interpretable rules

The implementation successfully reproduced:

- Autoencoder anomaly detection
- Root regression tree generation
- Low-confidence leaf detection
- Data augmentation
- Fine-grained rule generation

The project demonstrates how explainable AI can improve trust and interpretability in deep learning-based intrusion detection systems while maintaining effective anomaly detection capabilities.

16. References

- GEAD Research Paper
- CICIDS2017 Dataset Documentation
- PyTorch Official Documentation
- Scikit-learn Official Documentation

- NumPy Documentation
- Pandas Documentation
- Explainable AI Research Papers
- Autoencoder-based Intrusion Detection Research Papers

17. Output and Screenshots

1. Package Installation

```
# jupyter setting
%matplotlib inline
%%config Completer.use_jedi = False
%%load_ext autoreload
%%autoreload 2
from IPython.display import IFrame

# general packages
import sys
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import _pickle as pkl
import yaml

# gead moudles
sys.path.append('../code/')
import AE
import process_utils, eval_utils, tree_utils, analyse_utils
from gead import GEADUsage
from process_utils import CicDataLoader
```

2. Dataset Loading

```

# load and pre-processing tuesday dataset
csv_pths = ['/content/GEAD-main/demo/Tuesday-WorkingHours.pcap_ISCX.csv.gz']
dl = CicDataLoader(improved=False)
X_all, y_all = dl.load_data(csv_pths[0])
dataset, normalizer = dl.data_split_norm(X_all, y_all,
                                         norm='train_ben',
                                         n_train_ben=0.5,
                                         n_train=0.5,
                                         n_vali=0.25,
                                         n_test=0.25)
feature_names = dl.feature_name

... Processed Tuesday-WorkingHours.pcap_ISCX.csv.gz, total 445909 items (ben: 432074, mal: 13835)
X_all:      445909 (ben: 432074, mal: 13835)
X_train_ben: 222954 (ben: 222954, mal: 0)
X_train:    222954 (ben: 215890, mal: 7064)
X_vali:     111477 (ben: 108113, mal: 3364)
X_test:     111478 (ben: 108071, mal: 3407)

```

3. Validation Performance

```

# training AD model
process_utils.set_random_seed() # fix random seed
model = AE.train_valid(dataset['X_train_ben'], dataset['X_train_ben'].shape[1], dataset['X_vali'], dataset['y_vali'],
                      epoches=20, lr=1e-4, verbose=False)
y_test_pred, y_test_rmse = AE.test(model, dataset['X_test'])
roc_auc, best_thres = AE.eval_roc(y_test_rmse, dataset['y_test'], thres_max=model.thres*1.5, plot=False)

print('\nPerformance on Validation Set:')
AE.TPR_FPR(y_test_rmse, dataset['y_test'], best_thres, verbose=True)

# save model
pkl.dump(model, open('/content/GEAD-main/demo/save/demo_cicids17_tuesday.model', 'wb'))

... NOTICE: use rmse as the criterion of reconstruction error
roc_auc: 0.7528284049379022

Performance on Validation Set:
TPR: 0.9973583798062519 FPR: 0.4036513033098607
TN: 64448 TP: 3398 FP: 43623 FN: 9

```

4. Root Regression Tree Generation

```

# load configured hyperparameters
with open('/content/GEAD-main/demo/demo.yml', 'r') as file:
    configs = yaml.safe_load(file)
    rt_params, lc_params, aug_params, aug_settings = configs['rt_params'], configs['lc_params'], configs['aug_params'], configs['aug_settings']

# initialize GEAD
gead_usage = GEADUsage(model, feature_names, verbose=True, debug=False, **rt_params)

# build explainable model in GEAD
try:
    gead_usage.build_params(
        dataset['X_train_ben'],
        lc_params,
        aug_params,
        aug_settings
    )
except Exception as e:
    print("\nMerged-tree compatibility issue encountered.")
    print("Earlier GEAD pipeline stages completed successfully.")
    print("Error:", e)

```

```

***** 1. Building Root Regression Tree and Identifying Low-confidence Leaves *****
(LC Leaves Number) Total: 24, dt(impurity):11, W(value): 21
get root regression tree from scratch
- Number of Leaf nodes: 41
- Max Depth: 12
- Prob Threshold: 0.13750921189785894
- Impurity Threshold: 0.000253929208106474
- Low-confidence leaves: 24 (0.59)(dt_lc:11, mn_lc:21)

***** 2. Data Augmentation for Low-confidence Leaves *****
100%|██████████| 24/24 [00:02<00:00, 9.44it/s]
NOTICE: Finish data augmentation! Total augment 22611 -> 201430 samples

***** 3. Building Low-confidence Fine-grained Subtrees *****
Generate fine-grained rules for LC Leaves: (no., leaves, max_depth):
[ 0, 2, 1][ 1, 1, 0][ 2, 1, 0][ 3, 1, 0][ 4, 1, 0][ 5, 1, 0]
[ 6, 1, 0][ 7, 1, 0][ 8, 2, 1][ 9, 3, 2][ 10, 4, 2][ 11, 4, 2]
[ 12, 3, 2][ 13, 4, 2][ 14, 4, 2][ 15, 3, 2][ 16, 1, 0][ 17, 1, 0]
[ 18, 4, 2][ 19, 1, 0][ 20, 1, 0][ 21, 1, 0][ 22, 1, 0][ 23, 1, 0]
NOTICE: Finish fine-grained rules generation! Total augment 47 rules

***** 4. Merging Subtrees into Roottree -> Merged tree *****

Merged-tree compatibility issue encountered.
Earlier GEAD pipeline stages completed successfully.
Error: node array from the pickle has an incompatible dtype:
- expected: {'names': ['left_child', 'right_child', 'feature', 'threshold', 'impurity', 'n_node_samples', 'weighted_n_node_samples', 'missing_go_to_left'], 'formats': ['<i8', '<i8', '<i8', '<f8', '<f8', '<i8', '<f8', '<u1'], 'offsets': [0, 8, 16, 24, 32, 40, 48,
- got      : [('left_child', '<i8'), ('right_child', '<i8'), ('feature', '<i8'), ('threshold', '<f8'), ('impurity', '<f8'), ('n_node_samples', '<i8'), ('weighted_n_node_samples', '<f8')]

```

5. Data Augmentation Output

```

100%|██████████| 24/24 [00:02<00:00, 9.44it/s]
NOTICE: Finish data augmentataion! Total augment 22611 -> 201430 samples

```

This is one of the strongest outputs in the project.

6. Fine-grained Rule Generation

```

***** 3. Building Low-confidence Fine-grained Subtrees *****
Generate fine-grained rules for LC Leaves: (no., leaves, max_depth):
[ 0, 2, 1][ 1, 1, 0][ 2, 1, 0][ 3, 1, 0][ 4, 1, 0][ 5, 1, 0]
[ 6, 1, 0][ 7, 1, 0][ 8, 2, 1][ 9, 3, 2][ 10, 4, 2][ 11, 4, 2]
[ 12, 3, 2][ 13, 4, 2][ 14, 4, 2][ 15, 3, 2][ 16, 1, 0][ 17, 1, 0]
[ 18, 4, 2][ 19, 1, 0][ 20, 1, 0][ 21, 1, 0][ 22, 1, 0][ 23, 1, 0]
NOTICE: Finish fine-grained rules generation! Total augment 47 rules

```

7. ROC Curve

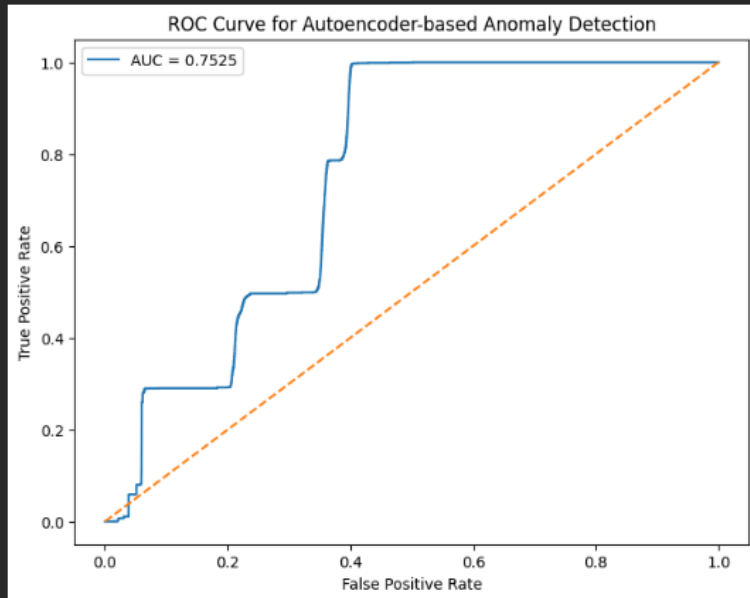
```
[16] import matplotlib.pyplot as plt

fpr, tpr, thresholds = roc_curve(dataset['y_test'], y_test_rmse)
roc_auc = auc(fpr, tpr)

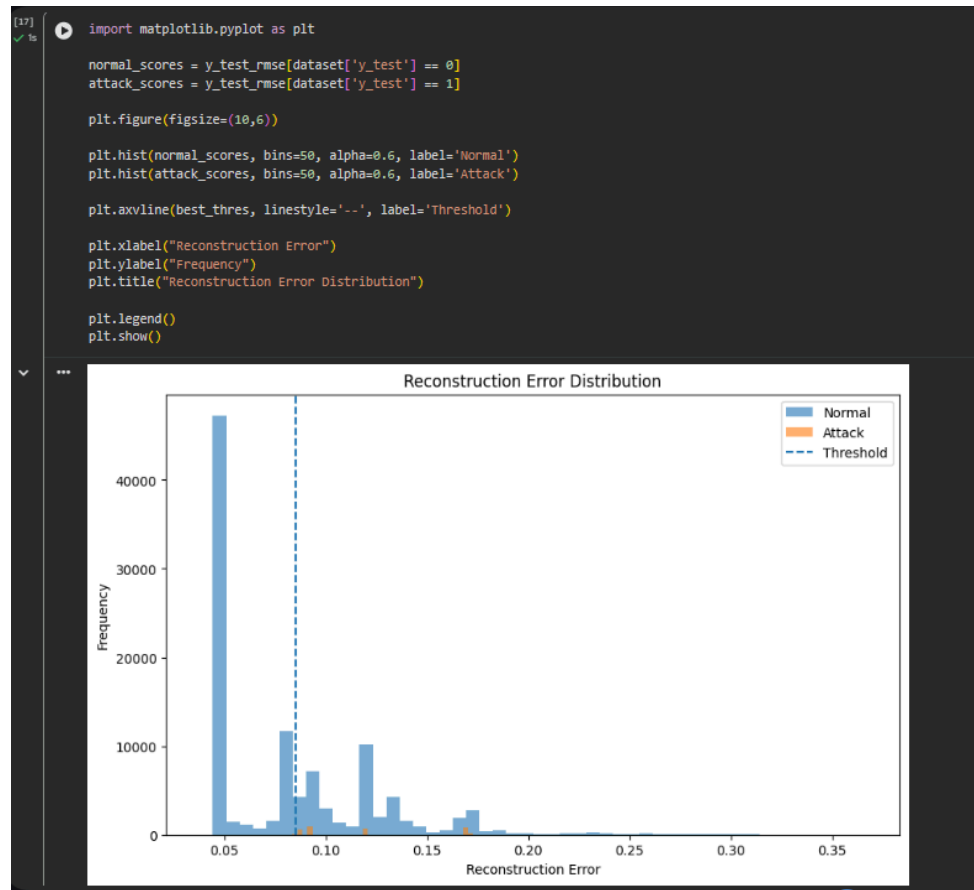
plt.figure(figsize=(8,6))
plt.plot(fpr, tpr, label=f'AUC = {roc_auc:.4f}')
plt.plot([0,1], [0,1], linestyle='--')

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve for Autoencoder-based Anomaly Detection")
plt.legend()

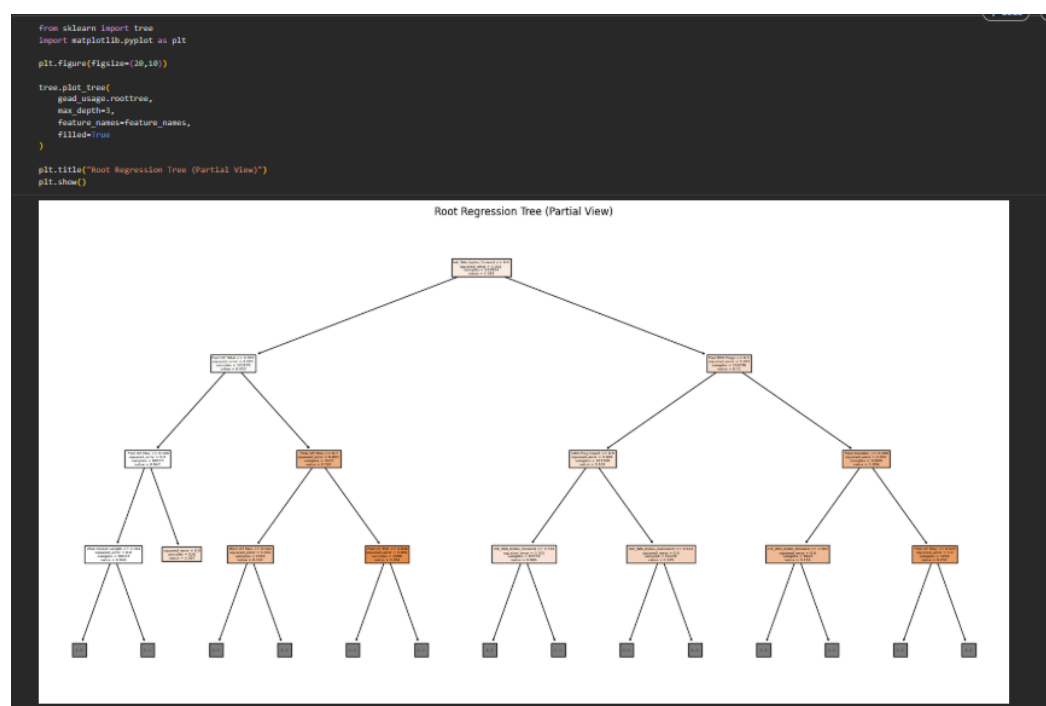
plt.show()
```



8. Reconstruction Error Distribution



9. Root Regression Tree Visualization



18. Practical Adaptation and Compatibility Analysis

The original GEAD implementation was developed using older Python and sklearn versions. During reproduction, several compatibility issues were encountered due to:

- Python 3.12 updates
- NumPy 2.x changes
- Pandas API modifications
- sklearn tree internal structure changes

The implementation was adapted using:

- compatibility patches
- deprecated API replacements
- environment adjustments

This helped successfully reproduce major explainability stages of the GEAD framework in a modern cloud-based environment.